

Cross-Level Access Verification: Confirming Governance Correctness Across Architectural Levels

Derivation Note B2.97 — Closing the B1.19 Decomposition

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026). It does not introduce new axioms. Its contribution is to articulate, in operational form, how cross-level access verification functions as the governance-correctness confirmation that closes the B1.19 cross-level access decomposition.

Abstract

Cross-level access in the Coordination Knowledge Substrate (CKS) architecture — the governed interaction paths between the Self, aspect, and cell levels that B1.19 names — requires not only governance design (B2.95) and pattern configuration (B2.96) but also operational verification that governance is correctly instantiated. This note formalizes cross-level access verification as that confirmation: the operational process by which a CKS deployment confirms that access rules are complete and substrate-resident, that access authority is correctly distributed, that no access events exceed authority boundaries, and that configured access patterns are functional. Verification covers four dimensions: access rule completeness per the A5.04 rule authoring test, authority distribution correctness per A2.47, unauthorized access detection via the A5.08 provenance-completeness test and A5.09 four accountability questions with violations registered as A1.03 first-class conflicts, and pattern functional integrity confirming that the expression mechanism, vertical evolution paths, and governance override are operational. Cross-level access verification closes the B1.19 decomposition cycle. It is architecturally distinctive because conventional AI architectures lack inter-component access governance to verify; the unauthorized access detection dimension is particularly novel in actively checking whether access events exceed explicitly assigned authority boundaries.

1. Why Cross-Level Access Verification Requires Standalone Formalization

Cross-level access in CKS is a governed architectural capability, not an assumed structural property. B1.19 establishes that the enterprise brain Self permits access between levels — downward from Self to aspect to cell, and upward from cell to aspect to Self — where purpose requires it, and that such access operates under governance rather than as unrestricted inter-component communication. B2.94 established the integrating frame for this decomposition: cross-level access is purposive, governed, and pattern-configurable. B2.95 formalized how governance distributes access authority through

substrate-resident rules. B2.96 formalized the access patterns themselves — the expression mechanism pattern, vertical evolution patterns, and governance override pattern as configured interaction shapes.

What B2.94 through B2.96 do not cover is how a deployment confirms that governance is correctly instantiated. Governance design (B2.95) specifies what authority distribution should be. Pattern configuration (B2.96) specifies what access patterns should operate. Neither answers the verification question: is the authority distribution actually correct? Are the access rules actually complete? Are the configured patterns actually functional? Does any access event in the record exceed the assigned authority boundaries?

Cross-level access verification is the operational answer to these questions. It is the confirmation step that closes the governance cycle — the process by which the architecture certifies, at deployment and periodically thereafter, that cross-level access is correctly governed, fully recorded, and pattern-consistent. Without verification, governance design and pattern configuration remain architectural commitments that may or may not be correctly realized in any specific deployment.

The strategic prior-art role of B2.97 is to establish that this verification step — covering these four specific dimensions, triggered by these specific events, producing substrate-recorded results — is a formalized architectural commitment of the CKS pattern, not a deployment-specific practice. Naming it as a standalone operational variant closes the B1.19 decomposition and places the verification architecture in the prior-art record.

2. The Four Verification Dimensions

Cross-level access verification covers four dimensions. Together they confirm that access governance is complete, authority is correctly assigned, unauthorized access is detected, and patterns are operational.

Dimension 1: Access Rule Completeness (A5.04 Rule Authoring Test). The A5.04 rule authoring test, applied to cross-level access rules, verifies that every rule governing cross-level access was authored per A2.04 and is substrate-resident per A2.46. Completeness verification asks: are downward access rules present for every required governance operation — Self-to-aspect, aspect-to-cell, and Self-to-cell through expression? Are upward access rules present for every required reporting operation — cell-to-aspect and aspect-to-Self? Is every access rule inspectable per A2.01, meeting the substrate visibility requirement that the A1.01 governance commitment requires? A deployment that cannot produce a complete set of substrate-resident access rules for every cross-level path that its governance design requires has not satisfied the A5.04 test for access rules.

Dimension 2: Authority Distribution Correctness (A2.47 Verification). A2.47 specifies that cross-level access authority is distributed through substrate-resident specification, with each entity's access authority explicitly assigned rather than implied. Verification of authority distribution correctness confirms three properties: authority for every entity participating in cross-level access is specified in substrate; no entity holds implicit access authority derived from structural position alone; and where access authority distribution has changed, A6.06 compliance verification confirms the distribution was updated correctly. The A6.06 compliance check is the change-triggered variant of

authority distribution verification — it runs specifically when access authority is added, removed, or modified, confirming that the substrate correctly reflects the post-change distribution.

Dimension 3: Unauthorized Access Detection (A5.08 / A5.09 / A1.03). The A5.08 provenance-completeness test, applied to access event records, verifies that every recorded cross-level access event carries complete A2.40 provenance metadata — writer attribution, timestamp, antecedent reference, and rule reference. Provenance completeness is the precondition for the authorization audit that follows. The access audit applies the A5.09 four accountability questions to every access event record: who accessed (entity identity); what was accessed (the target content or substrate at the cross-level destination); when the access occurred (timestamp); and why — under what authority rule the access was executed. For each access event, verification checks whether the accessing entity holds authority per A2.47 for the access type and direction recorded. Any access event outside the authority distribution — an entity accessing a level or content type for which it holds no A2.47-assigned authority — constitutes unauthorized access. Unauthorized access events are not silently discarded or flagged as log entries. They are registered as A1.03 first-class conflicts in the substrate, available for governance resolution. This registration is what makes unauthorized access detection an architectural commitment rather than a monitoring practice.

Dimension 4: Pattern Functional Integrity (Per B2.96). Verification confirms that the cross-level access patterns B2.96 formalized are operational, not merely configured. The expression mechanism pattern (Pattern 3, deriving from B2.30): Self DNA activates and deactivates cell DNA as specified, and the activation/deactivation behavior is confirmed by access event records showing the expected expression operations. The vertical evolution patterns (Patterns 6 and 7): upward reporting paths from cell to aspect and aspect to Self are operational, producing access event records; downward expression paths from Self to aspect and aspect to cell are operational, producing access event records. The governance override pattern (Pattern 4): override access is correctly configured for entities holding override authority per A2.47, and access event records confirm override operations are executable. The pattern integrity test asks whether each configured pattern produces access events as expected; a configured pattern that produces no access event records in normal operation, or produces records inconsistent with its specification, does not confirm functional integrity.

Temporal Triggers. Verification runs at four points: at deployment initialization, confirming access rule completeness and authority distribution correctness before any cross-level access occurs; after access authority changes, as A6.06 compliance verification; at periodic governance reviews, confirming that access governance is maintained as the deployment evolves; and after detecting potential unauthorized access patterns in event records, running full unauthorized access detection to assess scope and register conflicts for governance resolution.

3. What Makes Cross-Level Access Verification Architecturally Distinctive

Conventional AI architectures — including those with multi-component or multi-agent structures — do not have inter-component access governance that requires verification. Components communicate, call each other, or share memory without explicit authority specification per component and without recorded access event provenance that could be audited against authority boundaries. There is nothing

to verify in the access-governance sense because there is no access governance architecture to be in or out of compliance with.

CKS cross-level access verification is architecturally novel in three respects. First, it confirms access rule completeness — there is a specific set of rules that must be present and substrate-resident, and verification can determine whether that set is complete. Incompleteness is a confirmable failure state, not merely a configuration decision. Second, it confirms authority distribution correctness — each entity's access authority is explicitly assigned, and verification can determine whether the distribution is correctly specified and current. Implicit authority assumptions are confirmable failures. Third — and most distinctively — it detects unauthorized access by actively auditing access event records against assigned authority boundaries and registering violations as first-class conflicts.

The unauthorized access detection dimension is distinctive precisely because it commits the architecture to treating every access event as an auditable record whose authority basis can be checked, and treating authority exceedance as a conflict requiring governance resolution rather than as an anomaly to be logged and forgotten.

4. Inherited Paper 1 Commitments

Cross-level access verification inherits six Paper 1 commitments as directly load-bearing.

A5.04 (rule authoring test) supplies the verification method for access rule completeness. Applied to cross-level access rules, A5.04 confirms authoring per A2.04 and substrate residency per A2.46 — the same two-part check that A5.04 applies to any orchestration rules in a CKS deployment, here specialized to access rules governing cross-level interaction.

A5.08 (provenance-completeness test) supplies the precondition check for unauthorized access detection. Without complete A2.40 provenance on access event records, the authorization audit cannot be performed with confidence — detection depends on recording completeness.

A5.09 (four accountability questions) supplies the audit method for unauthorized access detection. Who, what, when, why — the four questions applied to each access event record constitute the authorization audit that identifies events outside A2.47 authority boundaries.

A2.47 (authority distribution) is the reference against which unauthorized access detection checks event records. Every access event is audited against the authority distribution A2.47 specifies. Authority distribution correctness verification also runs directly against A2.47's substrate specification.

A1.03 (conflict-as-first-class) is what gives unauthorized access detection its governance consequence. Unauthorized access events are registered as A1.03 first-class conflicts — they enter the substrate as preserved conflict state, visible to governance, available for resolution through directed selection per B1.14. Detection that does not produce governance-visible conflict state does not satisfy the CKS commitment.

A1.01 (governance) and **A2.40 (provenance)** supply the framework conditions: all verification results are recorded per A2.40 with full provenance, and verification is itself a governed process under A1.01's authority architecture — humans with appropriate access can inspect, modify, or override verification

results, and the verification process operates under human-authored orchestration rules.

5. Unauthorized Access Detection as First-Class Verification

Unauthorized access detection warrants extended treatment because it is the most architecturally novel verification dimension and the one most likely to be misread as a monitoring practice rather than an architectural commitment.

The distinction is operational. A monitoring practice detects access anomalies and produces logs. An architectural commitment registers violations as substrate content in a specific form — A1.03 first-class conflicts — that the governance architecture treats with specific consequence. The difference is not in the detection act but in what follows: a log entry is informational; an A1.03 first-class conflict is authoritative substrate state that governance must address.

The access audit runs per access event record. For each recorded access event, the audit confirms: does the accessing entity hold A2.47-assigned authority for the access direction (downward or upward) and target level (Self-to-aspect, Self-to-cell, aspect-to-cell, cell-to-aspect, aspect-to-Self)? An event whose accessing entity holds no such authority for its recorded access type fails the audit. That failure is an unauthorized access event.

Registration as an A1.03 first-class conflict means the unauthorized access event enters the substrate with the same provenance requirements and governance visibility as any other first-class conflict. Governance reviews it, determines its source — mis-configuration of access rules, incorrect authority assignment, action taken outside authorized scope — and authorizes correction through directed selection per B1.14. The same directed selection mechanism that governs DNA evolution applies to cross-level access governance corrections.

Detection depends on recording completeness. An access event that was not recorded cannot be detected as unauthorized. This is the principal limit on unauthorized access detection: the commitment to detect is only as strong as the commitment to record. A5.08's provenance-completeness test, applied to access event records, is the verification that the record is complete enough to support the audit.

6. Operational Implications

At **deployment initialization**, cross-level access verification confirms three properties before any cross-level access operations occur: access rules are complete per Dimension 1, authority distribution is correctly specified per Dimension 2, and configured patterns are demonstrably functional per Dimension 4. A deployment that fails initialization verification has not yet satisfied the B1.19 architectural commitment, regardless of whether access patterns are configured.

Authority changes trigger A6.06 compliance verification on the authority distribution. When an entity gains or loses cross-level access authority, A6.06 verification confirms the substrate reflects the changed distribution correctly. This triggered verification prevents access governance from drifting out of sync with intended authority design during the deployment's operational life.

Periodic governance reviews include cross-level access verification as part of the governance health assessment. At each review, unauthorized access detection runs across the accumulated access event record since the last review period, and any unauthorized access conflicts registered since the last period are surfaced for governance resolution. Pattern functional integrity is reconfirmed, particularly for vertical evolution paths that may be affected by structural reorganization under horizontal and vertical evolution per B1.14.

Cross-partner authority verification per A2.47 applies when the access authority distribution includes entities from multiple partners or organizational units. A2.47's requirement that authority be explicitly specified and substrate-resident applies uniformly; cross-partner access authority receives the same initialization and triggered verification as intra-organizational access authority.

Governance corrects unauthorized access through directed selection. When unauthorized access conflicts are registered, governance's response is human-directed correction of whatever produced the unauthorized event: access rule amendment, authority redistribution, or substrate correction as appropriate. The correction is itself a directed selection event, human-governed and substrate-recorded.

7. Limits

Cross-level access verification does not verify that access patterns are optimal — it verifies that they are governed and functional. Whether a particular authority distribution is appropriate for the deployment's purpose, or whether a configured pattern produces useful outcomes, is outside the scope of verification. Verification confirms governance correctness, not governance wisdom.

Unauthorized access detection depends on recording completeness. Where access event records are incomplete, detection is incomplete. The architecture commits to recording completeness through A5.08 and A2.40, but recording completeness itself must be verified rather than assumed.

Verification does not prevent unauthorized access — it detects it post-occurrence. The CKS commitment is to detection and governance resolution, not to enforcement at execution time. A system in which unauthorized access is impossible by architectural constraint — through enforcement mechanisms that the architecture itself does not specify — would be a different architectural commitment from what CKS formalizes.

Pattern functional integrity verification confirms that patterns produce access events as expected, not that they produce optimal outcomes. A pattern that operates as configured but is poorly configured produces access events that pass functional integrity verification while failing to serve the deployment's governance purposes. Configuration quality is a governance question; pattern functionality is what verification can confirm.

Cross-level access verification closes the B1.19 decomposition cycle: integrating frame (B2.94) → governance (B2.95) → patterns (B2.96) → verification (B2.97). Each note established a necessary component of the cross-level access architecture; verification is what confirms the components are correctly assembled and operating as the governance design intends.

8. Architectural Verification Test

A CKS deployment satisfies the cross-level access verification commitment if and only if all four of the following hold: (1) the A5.04 rule authoring test, applied to all downward and upward access rules, confirms that every required cross-level access rule is substrate-resident and authored per A2.04; (2) A2.47 authority distribution is completely and explicitly specified for all entities with cross-level access, with no implicit authority assumptions, and A6.06 compliance is confirmed for any changes since last verification; (3) the A5.08 provenance-completeness test confirms all access event records carry complete A2.40 provenance, the A5.09 four accountability questions yield determinate answers for any access event, and any access event outside A2.47 authority boundaries is registered as an A1.03 first-class conflict; (4) each configured cross-level access pattern produces access event records confirming the pattern is operational.

9. Why Naming This as Standalone Matters — and the B1.19 Decomposition Closed

Naming cross-level access verification as a standalone operational variant establishes that the verification step — as a four-dimension governance confirmation with specific inherited commitments, specific temporal triggers, and a specific relationship between unauthorized access detection and first-class conflict registration — is an architectural commitment of the CKS pattern, not a deployment recommendation. Without this formalization, the prior-art record contains B1.19, B2.94, B2.95, and B2.96 but not the explicit commitment to how correctness is confirmed. A party implementing cross-level access governance without unauthorized access detection, or without registering violations as first-class conflicts, or without the four-dimension verification architecture, would occupy unformalized territory. B2.97 forecloses that territory.

The B1.19 decomposition is now complete. B2.94 established that cross-level access is purposive and governed — the integrating frame. B2.95 established how governance distributes access authority through substrate-resident rules. B2.96 established the access patterns through which cross-level access operates. B2.97 establishes how the architecture confirms that access is correctly governed, recorded, and pattern-consistent. The four notes together decompose the B1.19 commitment from design through implementation to verification — the full governance cycle for cross-level access in a CKS enterprise brain Self.

Phase B2 continues with B2.98, which begins the B1.20 recursive Paper 1 commitments decomposition — the final Phase B2 decomposition, covering the six architectural advantages where CKS exceeds biological analogy as formalized in Paper 2’s §2.2 and §9.

Source Papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

How to Cite This Note

Li, W. (2026). Cross-Level Access Verification: Confirming Governance Correctness Across Architectural Levels (Derivation Note B2.97). May 12, 2026. ORCID: 0009-0004-8065-3235.